

Correction-Aware Data Admission: Candidate-Bound Authorization and Field-Level Provenance for AI-Derived Updates

Liang Hanghao^a, Xuan Wentao^a, Chen Qile^a, Peng Peng^{a,*}

^aCollege of Computer Science and Electronic Engineering, Hunan University, Lushan South Road, Yuelu District, Changsha, 410082, China

Abstract

AI-assisted data acquisition separates a machine proposal from the value that a human authorizes for a database. A correction can preserve the final value while obscuring which proposal was reviewed or replacing the sources of unchanged fields. We specify a correction-aware admission contract that binds an exact persisted candidate to a separately attributable authorized value, checks predecessor freshness, and atomically constructs a complete successor with total field-level provenance. Five paired histories establish class-wise irredundancy relative to explicitly defined observations. A Python/SQLite realization connects the contract to bounded relational checks, persisted-state projections, and transaction tests. A new 495-execution comparison evaluates a context-preserving event journal, its exact-binding extension, and the reference implementation. The latter two agree on all 165 corresponding cases, showing that the contract can be implemented by different storage designs. In 60 scripted browser cases, an experimental review-session gate blocks nine request substitutions accepted by the original confirmation path; both paths accept three display-only substitutions, delimiting the trusted review boundary. On the repaired implementation, 22,400 timed observations characterize admission and provenance reconstruction. At the largest trace-grid point, bulk access has a median of 170.24 ms versus 509.09 ms for point access with the same verifier. These constructed workloads support the specified integrity obligations and implementation behavior, without establishing universal minimality, display integrity, or deployment-wide performance.

*Corresponding author. Email address: hnu16pp@hnu.edu.cn

Keywords: data integrity, data provenance, human authorization,
AI-assisted data acquisition, transactional admission, relational verification

1. Introduction

An AI-assisted data acquisition system may extract a quantity from a document, propose a correction to a record, and ask a person to approve the update. Once admitted, that value can be used by queries, reports, and subsequent decisions. The database must then answer more than what its current value is: which persisted proposal was reviewed, what value the reviewer authorized, which record version that authorization concerned, and where the other fields of the resulting record came from. This paper studies the integrity constraints needed to preserve those answers when a machine proposal can be corrected before admission.

Consider a record whose quantity is 90. A recognizer proposes 100, but a reviewer authorizes 101; an unchanged batch-code field should retain its earlier source. Replacing the stored proposal with 101 would erase the distinction between machine output and human correction. Recording only the final value and reviewer would retain value authority while leaving the reviewed proposal ambiguous. Creating a new snapshot without copying the batch-code source would preserve the record’s values while breaking its field-level provenance. The required relation therefore retains the predecessor, the immutable proposal x_c , and the authorized value x_a , including the case

$$x_c \neq x_a. \tag{1}$$

Candidate identity presents a separate policy choice. Two extraction attempts may propose the same value for the same field and evidence. A policy concerned with approved values can treat them as interchangeable. An instance-level accountability policy must distinguish them if later review must identify the particular proposal selected by the reviewer. We do not assume that every application needs this stricter policy. We characterize its consequences and compare it explicitly with a context-rich policy that omits exact candidate identity.

Transactions, concurrency control, and provenance provide the foundations for implementing this requirement. Atomic transactions support consistent state changes; optimistic validation addresses conflicts with intervening updates; provenance models describe derivation and responsibility (Gray, 1981;

Kung and Robinson, 1981; Moreau and Missier, 2013). Our contribution is an application-level admission contract built using these mechanisms. It specifies their joint obligations when proposal origin and value authority differ. The relation is

$$\begin{aligned}
 & \textit{exact candidate} \rightarrow \textit{candidate-bound authorization} \\
 & \rightarrow \textit{explicit authorized value} \\
 & \rightarrow \textit{complete successor and total field sources.}
 \end{aligned} \tag{2}$$

The candidate remains historical evidence. Authorization supplies permission for the committed value, and a trusted transaction creates the successor. This relation can be represented by relational tables or an event journal; the contract does not privilege either storage architecture.

The central question is which distinctions must remain observable to check this relation against the failures considered here. We identify five information classes: candidate identity and context, separate proposed and authorized values, freshness, batch/successor completeness, and total field-source attribution. Paired histories show that removing each class under the specified observation projection can make a compliant history indistinguishable from a violating one. This is a conditional distinguishability result for a declared failure model, rather than a universal lower bound on all possible schemas, encodings, or authorization policies.

We instantiate the contract in AUTO-DECTE, a Python/SQLite reference realization, and make three contributions:

- C1: An admission contract and conditional distinguishability analysis.** We define candidate-bound authorization with separate proposal and authorized values, one complete successor, and exact propagation of changed- and unchanged-field sources. Five paired-history constructions establish class-wise irredundancy relative to the stated observations and failures.
- C2: Relational and transactional enforcement.** We connect the contract to an Alloy model, explicit relational constraints, a version compare-and-swap, and in-transaction validation. Selected formal-concrete projections and persisted-state checks examine whether the implementation preserves the modeled distinctions, including canonical JSON value equality.
- C3: Comparative, boundary, and cost evidence.** We compare the reference with independently implemented event-journal controls, exercise

the browser-to-authorization boundary, and measure admission, trace reconstruction, and storage on the current repaired implementation. The artifact includes case-level decisions, database states, query answers, and timing observations.

The new comparisons deliberately test where stronger controls close earlier gaps. Across 165 corresponding constructed cases, an event journal with exact candidate binding agrees with the reference on every admission decision and provenance-query answer. Its context-only variant accepts 15 same-value candidate substitutions that satisfy the value-level policy but violate the instance-level policy. These results locate the difference in the enforced relation, not in the use of an event log. They also expose the additional rejections imposed by choosing the stricter policy.

The review experiment establishes another limit. A server-held review binding blocks the tested request substitutions, but both evaluated paths accept cases in which only the page display is substituted while the server and request consistently refer to another candidate. Database integrity cannot establish what a person actually saw. The contract therefore assumes an authenticated caller and a trustworthy review channel; it records responsibility without proving factual truth, reviewer attention, or display integrity. The experiments use constructed histories and a local browser fixture, not an independent deployment or user study. Repeated timings characterize one implementation and machine rather than model accuracy or general operational prevalence.

The remainder presents the contract, its bounded relational characterization and transactional realization, the formal-concrete checks, and the comparative evaluation. Figure 1 summarizes the admission boundary.

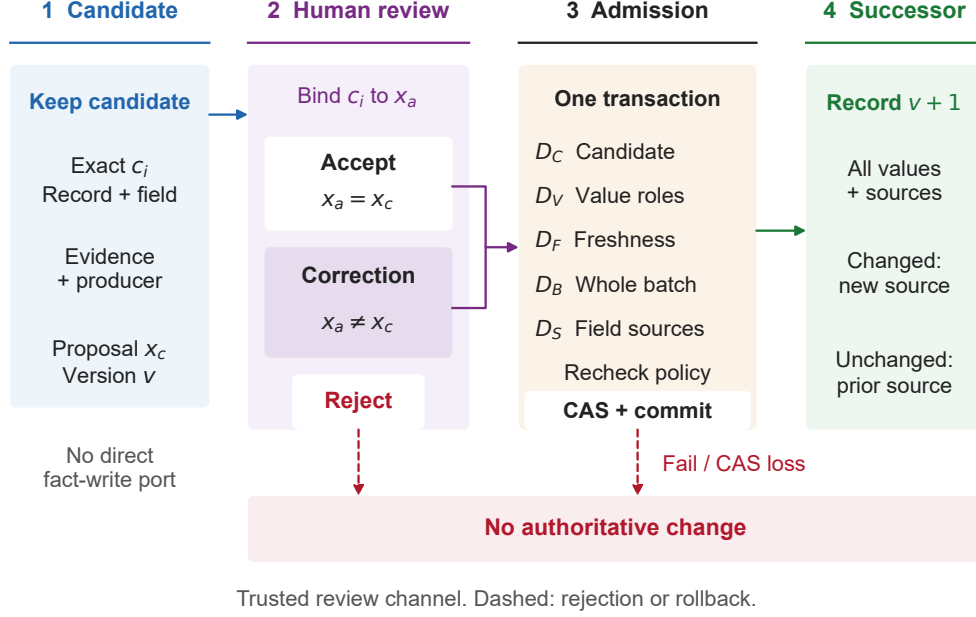


Figure 1: Correction-aware admission to authoritative data. Candidate-bound Accept or Correction decisions proceed to transactional validation. Version compare-and-swap protects creation of a complete successor and total field-source map. Rejection, failed validation, or a lost compare-and-swap leaves authoritative state unchanged. The authenticated review channel is a trust assumption. Diagram source: deterministic vector rendering of the declared workflow; AI-assisted scripting with OpenAI Codex during the September 2026 revision.

2. Background and related work

The proposed contract combines data-integrity enforcement with provenance of proposal and authorization. We distinguish the contribution from the underlying mechanisms: transactions and version checks make a chosen transition safe to execute; provenance and event histories preserve information about its origin; the admission contract specifies which relationships must hold between those records for the selected accountability policy.

2.1. Transactions, constraints, and freshness

The transaction concept provides atomic and durable transformations that preserve application consistency (Gray, 1981). Optimistic concurrency control validates a transaction before installing its effects and handles conflicting concurrent work by aborting or retrying (Kung and Robinson, 1981). These

are established foundations. Our version compare-and-swap is an application-level freshness check within a database transaction, not a new concurrency-control algorithm or a replacement for the DBMS’s isolation and recovery mechanisms.

The relevant distinction is between executing a transition atomically and specifying the transition that is permitted. A transaction can atomically write a candidate identifier, an approval, a value, and a source map even when those entries refer to different review contexts. A foreign key can establish that a referenced candidate exists; the application must additionally express that it is the candidate to which the authorization applies. Likewise, a freshness check can establish that a predecessor version is current without deciding whether the proposed value or the separately authorized correction should be committed. These are observations about the particular integrity relation studied here, not limitations on the expressiveness of relational databases.

Our contribution at this layer is the specification and examination of cross-record obligations: candidate and evidence context must match, the committed value must equal the authorized value, the changed-field set must coincide with the transition set, and unchanged fields must retain their sources. Such obligations can be implemented using database constraints, application validation, or a combination. The reference distributes them across relational structure and a trusted transaction. The experiments examine that distribution and its failure behavior rather than claiming a novel transaction primitive.

2.2. Data provenance and transactional histories

Why- and where-provenance distinguish explanations for the presence of an output from the origins of its values (Buneman et al., 2001). The database-provenance literature further separates why-, how-, and where-provenance and studies their relationships and uses in query processing, maintenance, and debugging (Cheney et al., 2009). These distinctions motivate retaining more than a current value. Our field-source map is narrower than a general query-provenance algebra: it identifies durable transitions responsible for the fields of a committed record version. We do not claim to derive provenance for arbitrary relational queries.

PROV-DM provides entities, activities, agents, derivations, and responsibility relations, with extension points for application-specific information (Moreau and Missier, 2013). A proposal can be represented as an entity, review and admission as activities, and the producer and reviewer as agents.

Thus the relationships in our contract are compatible with an established provenance vocabulary. The additional question addressed here is operational: which bindings must be checked before admission, and which source relationships must remain total in the resulting snapshot? Vocabulary compatibility does not by itself establish that an implementation checks these conditions; conversely, our implementation is not an evaluation of PROV serialization or interchange.

Transactional provenance directly addresses how updates produce database states. Arab et al. model query and transaction histories with multi-version provenance and reconstruct past provenance through reenactment, using audit logs and access to earlier database versions (Arab et al., 2018). Their setting is broader in its treatment of database operations and concurrency. Our focus is the application-specific authorization preceding a state change: a machine proposal and a human-authorized correction have distinct roles even when an ordinary update history records the same final value. Transactional provenance can capture such data if the application records it; the admission contract specifies the association that should be recorded and enforced.

2.3. Event sourcing and policy-sensitive comparison

Event sourcing represents changes as a sequence of events and supports reconstruction of application state (Fowler, 2005). It is a natural implementation option for an admission history. Whether a journal can identify a reviewed proposal depends on its event schema and validation rules. An immutable sequence does not choose the semantic granularity of approval, but it can preserve exact candidate bindings when those bindings are included.

This observation shapes our comparison. Both new journal controls retain candidates, review context, versions, complete field sources, and transactional behavior. One allows value-level authorization under the retained context; the other additionally binds authorization to the exact candidate instance. The exact-binding journal and the reference agree throughout the tested workload. Accordingly, the comparison supports the significance of an explicit policy distinction and its enforceability in different representations. It does not show that event sourcing is weaker than the reference design, or that all applications should reject equal-valued substitutes. The policy determines which distinctions are required and which additional rejections are acceptable.

2.4. Recent governed-memory and activation contracts

Several recent preprints address closely related boundaries. Continuity Kernel separates candidate preparation from atomic activation and binds proposal identity, the exact predecessor, pre-state authority, evidence, lineage, and a complete accepted unit (He and Yu, 2026a). It explicitly permits relational, object-manifest, and replicated-log realizations. Its activation contract therefore already supplies the general proposal/authority separation and complete-successor principle; those ideas are not claimed as new here. Our narrower subject is a reviewed field candidate whose immutable proposed value can differ from the authorized value, together with exact changed- and unchanged-field attribution and the conditional observation analysis for that relation.

MemTxn places source-support admission, conflict-conditioned visibility, and complete application-state recovery outside the answer model (Cui et al., 2026). Its declared admission rule checks support in the referenced source, whereas our contract checks the relation between a persisted proposal, an attributable human authorization, and a committed correction. These are different validation predicates, and both may be needed in one application. MutMem binds signed retrieval-weight mutations to provenance, signer epochs, old/new weights, and a nonforking predecessor (Saidi, 2026). It establishes an adjacent authorization-and-continuity mechanism for mutation; we study human correction and field-source propagation rather than cryptographic authorization of adaptive retrieval weights.

LatticeMind represents claim status, conflict, and supersession at write time while retaining provenance (Zhou et al., 2026). Stored Is Not Supported takes an authenticated accepted-state head as input and governs the subsequent release of supported, current, disclosure-authorized assertions (He and Yu, 2026b). These works address claim selection and outward assertion respectively. Our admission contract does not replace either function. We cite all five as preprints and compare their stated boundaries, without claiming that their representations cannot encode our relation. The additional evidence here is the correction-specific paired-history analysis and executable comparison, including the result that a sufficiently bound event journal matches the reference implementation.

2.5. Authorization and the review boundary

OAuth DPOP binds token use to a proof-of-possession key and includes request-related checks such as the HTTP method and target URI (Fett et al.,

2023). It illustrates the need to specify precisely what an authorization mechanism binds. Request authentication and replay resistance do not themselves provide a claim about the content displayed to a reviewer. Our contract assumes host-provided identity and channel integrity; it does not supply a cryptographic authentication protocol. The browser experiment separately observes display state and the resulting request, exposing where an earlier server-held review binding helps and where a display-only substitution remains outside its guarantee.

AgentDojo evaluates tool-using agents over untrusted data and distinguishes task success from security failures (Debenedetti et al., 2024). Our evaluation addresses a narrower, deterministic boundary: whether a proposed write satisfies the admission relation. Archived model outputs provide some proposal values, but the new experiments make no model calls and do not estimate resistance to adaptive prompt injection. This separation avoids attributing host-enforced database checks to a model’s reasoning performance.

2.6. Relational analysis and executable checks

Alloy and Kodkod support bounded relational analysis using SAT-based model finding (Jackson, 2002; Torlak and Jackson, 2007). TestEra connects relational specifications with executable programs through concretization and abstraction (Marinov and Khurshid, 2001). We use these established methods to examine legal histories, constraint ablations, and selected persisted-state projections. A bounded absence of counterexamples is distinct from an unbounded proof; selected concrete projections do not establish full refinement of the application. The paired-history argument, bounded checks, database-state observations, and browser controls therefore support different parts of the contract. Their combination provides inspectable evidence while preserving the limits of each method.

3. Authoritative-state admission contract

3.1. Candidate, authorization, and authoritative state

Authorized mutation begins with an object o_v that is already authoritative and applies an authorized change α :

$$\text{Mutate}(o_v, \alpha) = o_{v+1}. \quad (3)$$

Correction-aware admission begins instead with an untrusted persisted candidate c , an attributable authorization a , and an authoritative pre-state Σ_v :

$$\text{Admit}(c, a, \Sigma_v) = \Sigma_{v+1}. \quad (4)$$

Admission does not make the candidate itself authoritative. It transforms a candidate-bound authorization into an authoritative successor while retaining the candidate as non-authoritative historical evidence. Proposal origin, value authority, and committed value are therefore distinct semantic roles.

Let r be a record with a nonempty authoritative field set F_r . Its committed state Σ_v contains a total value map $V_v : F_r \rightarrow X$ and a total source map $S_v : F_r \rightarrow T$, where T is the set of durable fact transitions. A machine candidate for field f is

$$c = \langle r, f, e, v_{\text{exp}}, x_c, p, c_{\text{id}} \rangle, \quad (5)$$

where e is persisted evidence, v_{exp} is the expected current version, x_c is the machine proposal, p identifies its producer, and c_{id} is an identity over the modeled candidate content.

Evidence identity is not content equality alone. The realization uses

$$\text{EID}(e) = \langle H(\text{bytes}(e)), L(r, f, u, \kappa) \rangle, \quad (6)$$

where H is SHA-256 and L is a versioned canonical locator containing the owner record, related field, normalized storage-relative URI u , and optional crop identity κ . The Alloy model abstracts content and locator as primitive domains. Its `certIdentityFacts` assumes unique certificate identities over the modeled certificate objects, excluding identity collisions from the checked scopes. Relating this abstraction to the implementation requires collision resistance of the deployed digest and integrity of the canonicalization and evidence store; it is not a proof or claim that SHA-256 is mathematically injective.

An attributable authorization is

$$a = \langle c_{\text{id}}, x_a, q \rangle, \quad (7)$$

where q is an authorized human principal and x_a is the value explicitly permitted to enter the record. The dual-value semantics are

$$\begin{aligned} \text{proposal}(c) &= x_c, & \text{authorize}(a) &= x_a, \\ \text{commit}(\Sigma_{v+1}, f) &= x_a, & \text{origin}(\Sigma_{v+1}, f) &= c. \end{aligned} \quad (8)$$

Accept has $x_c =_{\text{json}} x_a$. Correction has $x_c \neq_{\text{json}} x_a$; the candidate remains unchanged and the committed value derives its authority from a . Here origin identifies the retained proposal, not the authority for a corrected value.

Canonical value equality. Two values are identical throughout the contract only when their canonical JSON serializations agree. Write $x =_{\text{json}} y$ when the sorted-key, whitespace-minimal JSON forms of x and y are equal, and $x \neq_{\text{json}} y$ otherwise. Python equality would identify 2 with 2.0 and 1 with `true`; the contract does not, because a successor whose serialized value changes must obtain a new transition and source even when the two values compare equal in the host language. The changed-field domain, the admission planner, transition-value checks, reverse-trace validation, the independent database oracle, and the formal projection use this single relation; section 6 reports the regression that closed a cross-layer disagreement in an earlier build.

3.2. History, observations, and outcomes

An authority-relevant history is

$$h = \Sigma_0 \xrightarrow{e_1} \Sigma_1 \xrightarrow{e_2} \dots \xrightarrow{e_n} \Sigma_n, \quad (9)$$

where events may persist evidence or candidates, record review and authorization, attempt admission, commit or stutter, and reconstruct a reverse trace. The abstraction describes semantic information rather than requiring particular table or column names.

We group the observable distinctions into

$$\mathcal{D} = \{D_C, D_V, D_F, D_B, D_S\}, \quad (10)$$

where D_C is candidate identity and context, D_V is dual-value attribution, D_F is freshness, D_B is batch/successor completeness, and D_S is total source attribution. For $I \subseteq \mathcal{D}$, $\pi_I(h)$ retains only the information classes in I . Candidate identity may be implemented by a certificate, stable handle, or another equivalence class, provided it does not collapse to value equality. Freshness may likewise use a version, state hash, epoch, or comparable discriminator.

To make “retains” precise, let $\text{obs}_d(h)$ be the observation function for class d and define

$$\pi_I(h) = \langle \text{obs}_d(h) \rangle_{d \in I}. \quad (11)$$

Table 1: Observation functions used by the conditional characterization. “Derives” lists information available without consulting another class.

Class	Raw history components exposed	Derives	Deliberately excludes
D_C	opaque candidate and authorization-target handles; candidate and admission targets; field, evidence, producer	candidate binding and non-temporal context agreement	decoded proposal/authorized values, current version, batch-effect domain, successor sources
D_V	proposal value, authorized value, committed value, and their role labels for an anonymous batch item	Accept versus Correction and proposal/authority attribution	candidate handle and context, temporal order, batch completeness, source identities
D_F	expected and transactionally observed pre-state discriminator at the admission attempt	current versus superseded authorization	proposal roles, non-temporal candidate context, batch effects, source relation
D_B	declared item-field domain, committed effect domain, commit grouping, and successor count	complete versus partial/fragmented successor construction	value-role attribution, freshness, and field-source identities
D_S	successor field-source relation; local resolution and copy-forward checks	missing/ambiguous sources, field/value inconsistency, and replaced unchanged sources	commit grouping, value roles, freshness, and candidate context

The functions are logical projections, not claims that a database must store five physically separate objects. A certificate can co-locate several classes. In a projection, however, a content address is treated as an opaque equality-preserving handle: its preimage is not available as a side channel. This prevents a nominally omitted value or version from being recovered merely because the concrete certificate hash covers it. Table 1 defines what each function exposes and deliberately hides.

The full-history authority-outcome label is

$$O(h) = \langle status, successor, trace \rangle, \quad (12)$$

where *status* is admissible, inadmissible, or failed/stuttering; *successor* classifies the authoritative post-state or its absence; and *trace* is complete, incom-

plete, ambiguous, or unavailable. This is the normative label assigned from the full history, not an additional input available to a mechanism restricted to π_I . A binary accept/reject label is insufficient because value-identical successors may differ in source completeness.

An observation set I fails to distinguish a failure family when there exist a safe history h_s and unsafe history h_u such that

$$O(h_s) \neq O(h_u) \quad \wedge \quad \pi_I(h_s) = \pi_I(h_u). \quad (13)$$

Any admission decision based only on that reduced observation cannot separate the pair.

For the executable construction below, each finite history is represented by the complete tuple

$$\hat{h} = \langle r, C, A, B, v_{\text{exp}}, v_{\text{obs}}, F_{\text{decl}}, K, V_v, V_{v'}, S_v, S_{v'}, T \rangle, \quad (14)$$

containing the target, candidate and authorization registries, batch references, expected and observed pre-versions, declared fields, a commit sequence K , endpoint value and source maps, and a registry T of source-transition identifiers, fields, and values. Each commit step records a distinct successor identity and complete before/after value maps. The domain predicate checks unique candidate and authorization keys, resolvable batch references, schema-valid fields, total value maps, and a connected commit sequence from V_v to $V_{v'}$. Its committed effect domain is the union of the steps' actual value-change domains; its successor count is $|K|$. Neither is an independently assigned witness attribute.

Source observations expose each field's anchor count, reverse-resolution count in T , agreement of the resolved field/value with the successor, and exact prior-source retention for an unchanged field. Missing, duplicate, or inconsistent sources remain semantic failures rather than being excluded by the domain predicate. These are local source checks: the complete authorization-candidate-evidence chain belongs to the operational P6 checks below. Source observations hide intermediate commit grouping; freshness observes the admission-entry pre-state comparison. These explicit observation boundaries make it possible to compare atomic and fragmented histories with identical endpoints.

3.3. Conditional failure-distinguishability characterization

Table 2 summarizes the paired histories for this fixed observation basis.

Table 2: Failure-distinguishing information classes in the declared observation model.

Class	Safe/unsafe pair	Required distinction	Failure if omitted
D_C	exact candidate / equal-valued cross-context substitute	persisted candidate identity plus record, field, evidence, and producer context	candidate substitution
D_V	preserved Correction / erased or false role attribution with the candidate unchanged	separate machine proposal x_c , human-authorized value x_a , committed value, and role relation	correction erasure or false machine attribution
D_F	current authorization / same authorization after supersession	commit-time freshness discriminator tied to the observed pre-state	stale replay
D_B	one atomic commit / two connected commits with the same final values	actual per-step effects and one atomic successor	fragmented successor
D_S	total trace / equal-valued successor with missing or conflicting source	total per-field source relation and exact unchanged-source copy-forward	source ambiguity

Proposition 1 (conditional failure distinguishability). For each information class $d_i \in \mathcal{D}$, the declared failure model contains a safe history and a corresponding unsafe history whose authority outcomes differ but whose projections become equal when d_i is omitted. Consequently, an admission mechanism that observes only $\mathcal{D} \setminus \{d_i\}$ cannot distinguish the paired failure family under this model.

Proof sketch (by construction). For D_C , hold the value-role, temporal, batch, and source observations constant while changing non-temporal candidate context. For D_V , retain the same candidate, proposal value 100, and committed value 101, but falsely attribute the corrected value to the machine proposal. For D_F , change only the admission-entry freshness comparison. For D_B , compare one two-field commit with two connected single-field commits that reach the same final values and sources; the latter exposes an intermediate partial state. Effect domains and successor counts follow from these sequences. For

D_S , remove one source anchor while retaining candidates, values, freshness, and the commit sequence. Each construction changes its selected observation coordinate and the derived outcome, while leaving the other four coordinates equal.

Thus, for every $d_i \in \mathcal{D}$, the constructed histories satisfy

$$\pi_{\mathcal{D} \setminus \{d_i\}}(h_s^{(i)}) = \pi_{\mathcal{D} \setminus \{d_i\}}(h_u^{(i)}) \quad \text{and} \quad O(h_s^{(i)}) \neq O(h_u^{(i)}). \quad (15)$$

Any decision function of the reduced projection receives the same input for both histories and cannot assign both distinct required outcomes. This establishes the claimed conditional irredundancy.

The executable checker validates each complete $\hat{h}$ and derives observations and outcomes from its relations; witnesses store neither. It checks domain validity, unequal outcomes, equal four-class projections, and unequal full projections. Schema fields define the total value/source domains, whereas declared item fields define the required effect domain.

The original D_C pair changes record context while retaining the candidate handle, establishing irredundancy of the combined class. An additional pair fixes the two-candidate registry and every context/value component, then selects either the authorized candidate or the equal-valued alternative. It isolates candidate binding within this observation model. A single-field copy-forward control checks the distinction between schema and effect domains. Supplement S2 identifies the executable constructions and raw histories.

Scope of the result. Proposition 1 establishes class-wise irredundancy for the declared failure, history, observation, and outcome model, not universal minimality, schema uniqueness, or completeness over all admission failures. The checker validates the five class pairs, identity pair, and copy-forward control; it is not a general admission oracle. These constructions establish Eq. (15); the separate Alloy checks test encoded assertions and conjunct-removal sensitivity in finite scopes.

3.4. Batch admission relation

A batch B is a nonempty set of items sharing one target record, principal, expected pre-version, and successor. Under Full, item fields are unique. Let

$$\Delta(V_v, V_{v+1}) = \{f \in F_r \mid V_v(f) \neq_{\text{json}} V_{v+1}(f)\}. \quad (16)$$

For a post-initial concrete admission,

$$\Delta(V_v, V_{v+1}) = \{i.f \mid i \in B\} = \{t.f \mid t \in T_{v+1}\}, \quad (17)$$

with one transition per item. At the first certificate-era version, the batch covers all of F_r , establishing total value and source domains. Deletion is outside the model.

For every $i \in B$, admission requires

$$\begin{aligned} i.r &= r, & i.f &\in F_r, \\ EID(i.e) &= EID(i.c.e), & i.v_{\text{exp}} &= v, \\ i.a.c_{\text{id}} &= i.c.c_{\text{id}}, & i.a.q &= q, \\ i.x &=_{\text{json}} i.a.x_a, & t_i.x &=_{\text{json}} V_{v+1}(i.f) =_{\text{json}} i.x. \end{aligned} \quad (18)$$

Together these instantiate the admission preconditions

$$\begin{aligned} &\text{Bind}(a, c) \wedge \text{Fresh}(c, \Sigma_v) \wedge \text{Authorized}(a) \\ &\wedge \text{Complete}(\Sigma_{v+1}) \wedge \text{Attributed}(\Sigma_{v+1}). \end{aligned} \quad (19)$$

The successor is atomic: all admission-time decision, authorization, transition, version, source-map, and audit effects commit together, or authoritative state stutters. This does not require rollback of evidence, candidates, or review intent persisted before the admission attempt. For a changed field, $S_{v+1}(f) = t_i$. For an unchanged field, value and exact source are copied forward:

$$f \notin \Delta(V_v, V_{v+1}) \Rightarrow V_{v+1}(f) =_{\text{json}} V_v(f) \wedge S_{v+1}(f) = S_v(f). \quad (20)$$

Alloy permits same-value admission for correspondence with the historical singleton model; concrete post-initial admissions require a value change. For submitted items U , the admitted batch is $B = \{i \in U \mid i.x \neq_{\text{json}} V_v(i.f)\}$, with $B \neq \emptyset$. A mixed submission admits exactly B and copies unchanged items' exact prior sources into the complete successor, creating no transition or authorization binding for $U \setminus B$. Thus Eq. (17) concerns admitted effects, not all submitted reviews. Recording renewed review or re-attribution without a value change requires a separate review-event path outside this contract; persisted candidate identity allows that path to distinguish equal-valued proposals.

3.5. Operational properties and trust boundary

Model output and candidate-only callers are untrusted. The trusted computing base comprises the admission service, adapter, exercised database

transaction semantics, canonical serialization and hashing, and principal-policy source. Host authentication, session integrity, and faithful review presentation are external assumptions. In particular, the reference confirmation API constructs the binding from the caller-supplied candidate identifier and value; the database cannot infer what a person saw. The contract preserves and enforces the authorization thus constructed. It does not protect a compromised process, host, or database administrator.

Authorization timing and revocation. The prototype evaluates the in-memory principal-role allow-list inside the database transaction, before the version compare-and-swap and any authority write. A revocation that completes before this check causes zero-write rejection. The allow-list lock protects only the policy lookup/update; it is not held for the remainder of the database transaction. Consequently, a revocation that overlaps an admission after its successful policy check does not cancel that in-flight commit. The implementation therefore provides commit-entry revalidation, not linearizable revocation against already admitted operations, distributed identity-provider semantics, or session invalidation. Deployments requiring those properties must place a transactional policy epoch or equivalent revocation discriminator inside the admission compare-and-swap.

Human authorization is not a truth oracle. The contract establishes who authorized which persisted candidate and value in which context, and whether the successor was constructed atomically. It cannot establish that the authorized value is factually correct.

Table 3: Admission properties and their operational interpretation.

Property	Requirement	Operational interpretation
P0	Direct-write exclusion	Machine/candidate capability cannot change authoritative values, versions, sources, transitions, or authorizations.
P1	Candidate non-substitutability	Authority refers to the same canonical persisted certificate, not merely an equal value or analogous candidate.
P2	Context-bound admission	Record, field, persisted evidence content and locator, and expected version agree across the item and certificate.
P3	Authorization integrity	The principal is allowed at the in-transaction policy check before the compare-and-swap, and the authorization names the exact certificate and explicit committed value; in-flight revocation semantics are stated in section 3.5.
P4	Freshness	The expected version equals the transactionally observed current version; stale or competing decisions fail closed.
P5	Successor integrity	One atomic successor has an item-transition bijection, exact changed-field effects, and complete changed/unchanged framing.
P6	Trace soundness	Each committed field resolves through its exact source transition to consistent authorization, certificate, evidence, producer, and version relations.

4. Bounded relational characterization

4.1. Model structure

The Alloy model represents records, authoritative fields, linearly ordered versions, evidence content and locators, candidates, certificates and primitive identities, principals, authorizations, transitions, and states. A state maps each committed record/field pair to at most one value and one source transition. Once a record has an authoritative snapshot, both maps cover exactly its declared field domain.

Three event families define the bounded histories. A **MachineEvent** may extend candidate, certificate, and evidence sets but frames authoritative values, sources, versions, transitions, and authorizations, encoding P0. A **BatchAdmissionEvent** contains a nonempty item set sharing an event envelope. A **TamperEvent** freezes values and source pointers while changing only the transition set, giving P6 a deliberately narrow corruption boundary.

The base effect advances one record version and applies item values and sources while intentionally leaving transition structure underconstrained. Full supplies unique item fields; initial or later complete-snapshot shape; preexisting certificate and evidence; record, field, evidence-identity, evidence-owner, certificate-version, freshness, authorization-certificate, principal, and authorization-value bindings; and an exact item–transition bijection. Keeping these conjuncts separate permits one encoded distinction to be removed without silently changing the others.

4.2. From information classes to Alloy checks

Table 4 connects the observation classes of equation (10) to relational operators and executable evidence. Several conjuncts may encode one class; their ablations measure sensitivity of this encoding.

Each ablation pair contains at least two items: one violates only the selected conjunct and another remains Full-valid. The same attempted item set is admitted by the reduced relation and rejected with authoritative-state stutter by Full. This tests the sensitivity of the selected relational conjunct. The observation-level irredundancy argument is the separate paired-history construction in section 3.

4.3. Legal, unsafe, and preservation commands

Six legal-witness commands cover multi-field Accept, Correction, mixed Accept/Correction, same-value admission, initial snapshot, and singleton

Table 4: Mapping from failure-distinguishing information classes to bounded and concrete evidence.

Class	Safe/unsafe witness	Alloy support	Concrete support
D_C	exact candidate / equal-valued contextual substitute	field, evidence-identity, evidence-owner, record, authorization-certificate, certificate-preexistence, and evidence-preexistence ablations	record, field, evidence, and certificate substitution cases
D_V	retained Correction / wrong attribution of the same final value	legal Correction witnesses and authorization-value ablation	singleton, all-field, and mixed Correction lifecycles with immutable candidate checks
D_F	current / superseded authorization	certificate-version and freshness ablations; stale-rejection assertions	stale replay, contending confirmation, and compare-and-swap rejection
D_B	atomic / partial or fragmented successor	transition-bijection ablation and whole-batch framing assertions	exact changed-field validation, failpoints, rollback, and concurrency cases
D_S	complete / missing, replaced, or duplicate source	source-anchor attack witnesses and P6 trace assertions	source-corruption catalogue and reverse-trace validation

admission. Eleven paired ablations remove the conjuncts mapped in table 4. Principal membership remains fixed; the unused `ABL_7` marker is excluded because host identity trust is an external assumption.

Three source-attack commands start from a legal Full prefix and then remove a source anchor, replace it, or add a duplicate source-version transition; each requires an incomplete trace. Fourteen assertions run per scope. Nine guard definitions and immediate consequences: P0/P1/P3/P4/P5, whole-batch effects, rejection framing, and two P6 checks. Five check invariant preservation, two singleton correspondences, rejection of an ablated attempt, and positive trace completeness under a well-formed pre-state.

The positive assertion requires a trace-complete predecessor and no stray transition to the successor version; otherwise the permissive base effect admits a post-state that fails `traceComplete`. It covers admitted fields in one step, not arbitrary histories or every carried-forward field. Both profiles include reachable initial and non-vacuous carry-forward witnesses for the

Table 5: Bounded Alloy outcomes in the two declared scopes. SAT denotes a found witness; UNSAT denotes no instance within the encoded scope. Trace SAT counts witnesses to the antecedent of the conditional positive trace assertion.

Profile	Legal SAT	Ablation SAT	Attack SAT	Trace SAT	Total SAT	Total UNSAT	Commands
S1	6	11	3	2	22	14	36
S2	6	11	3	2	22	14	36

precondition. Removing the post-state certificate binding changes UNSAT to SAT. These are separate checks of reachability, bounded preservation, and sensitivity, rather than fourteen independent proofs. Concrete enforcement additionally validates predecessor prefixes and uses transition uniqueness and in-transaction CAS revalidation.

4.4. *Scopes and interpretation*

The S1 profile bounds principal structural domains at two records, three fields, six versions, up to six admission items/transitions for most commands, and four states/events. S2 increases representative bounds to three records, four fields, eight versions, up to eight admission items/transitions, and five states/events. Individual commands adjust Value, Evidence, or Transition bounds when a witness needs an extra atom. The artifact contains the executable scopes; this summary is not a replacement for them.

An UNSAT assertion means that the Analyzer found no counterexample within the encoded command scope. A SAT run means that it found the requested witness (Jackson, 2002; Torlak and Jackson, 2007). Singleton contract/effect checks additionally guard the batch lift against silently changing the historical one-item semantics.

5. Transactional realization

5.1. *Persistent objects and authority interfaces*

AUTO-DECTE implements the contract in Python, SQLAlchemy, and SQLite. Three application interfaces separate authority: `CandidateWritePort` appends evidence and candidates, `AuthorityReadPort` loads persisted objects, and `FactAdmissionPort` admits a complete transition bundle. Recognition receives the candidate facade; trusted review code receives the

admission facade. This is application-level capability separation within one process, not isolation from malicious code with database credentials.

A candidate certificate records its target record and field, canonical proposal, producer identity/version, selection artifact, expected fact version, evidence hash, and canonical locator. A human-decision row names the reviewer and candidate; an immutable authorization-binding row fixes the decision, exact certificate, and authorized value. A transition preserves this chain under both Accept and Correction. The principal policy is a revocable, thread-safe in-memory role mapping; authentication and session integrity belong to the host.

The reference `ReviewForms.confirm` API receives certificate identifiers, values, and a principal identifier from its caller and constructs decision/binding objects at confirmation. It validates the resulting database relations but has no independent record of the preceding display. Consequently, exact binding inside the transaction protects the constructed authorization, not the fidelity of the human interaction that supplied it. The experimental review-session gate in RQ2 tests an additional earlier binding boundary and retains the trusted-display assumption.

5.2. Evidence identity and admission transaction

Certificate construction, admission, and trace use the same canonical locator rules: NFC and path/URI normalization, rejection of unsafe path segments, and binding of record, field, storage-relative URI, locator version, and optional crop. Admission reloads the evidence row and compares owner, content hash, and reconstructed locator with the certificate; copying a locator string cannot conceal inconsistent persisted ownership. The artifact specifies the normalization rules.

Application-level preflight checks provide useful errors, but the database adapter repeats authoritative checks inside the transaction:

1. reload the declared field domain, current version, previous complete values and sources, certificates, evidence, and principal policy;
2. derive initial coverage or the later changed-field set under canonical JSON equality; reject empty later changes, deletions, hidden changes, duplicate fields, extra transitions, or an incomplete predecessor;
3. validate unused identities and the decision-binding-certificate-evidence-transition relation, including $x_{\text{authorized}} =_{\text{json}} x_{\text{transition}} =_{\text{json}} x_{\text{successor}}$;

4. conditionally update the record only if its current version equals the expected version;
5. create one transition per admitted field, retain exact previous sources for unchanged fields, and check identical value/source domains;
6. insert the decision, binding, transitions, complete version, field projections, and audit event in one commit.

The compare-and-swap is the first authoritative write. A validation error, unsuccessful CAS, or exception before commit rolls the admission transaction back. Unique indexes on decision/certificate bindings and record/version/field transitions, together with foreign keys, reinforce these checks. Rejection framing is relative to the state immediately before admission; preparatory evidence, candidate, or review records may already exist. In the new comparator experiment, zero-write checks begin after authorization preparation.

Each connection enables `foreign_keys=ON` and `busy_timeout=5000`. The application does not explicitly select an isolation level; its exercised configuration uses SQLite’s default deferred transactions through SQLAlchemy. Journal and synchronization settings are reported with each experiment because the admission and storage workloads use different settings. Porting the contract requires checking the destination DBMS’s transaction, locking, and conflict behavior.

The prototype also supports manual entry by deriving a certificate at confirmation. That compatibility path is excluded from claims about preexisting machine candidates and does not replace Correction. A separate historical agent-harness adapter exposes only proposal and verification operations, never confirmation; Supplement S5 records its integration evidence.

5.3. Reverse trace and source evolution

The query path checks every declared field in every prefix version. An initial field needs one transition created in that version. A later changed field needs its unique new transition; an unchanged field retains its exact prior source. The verifier checks the source record, field, creation/version identifiers, committed value, decision, authorization binding, certificate, producer, evidence hash/locator, and expected-version relations.

A trace is **complete** only when all checked fields and prefix relations agree. Missing or inconsistent rows yield **incomplete** with reasons; the query neither repairs data nor substitutes another source. Legacy states predating

the certificate scheme are identified separately. Trace completeness establishes consistency of the recorded authorization lineage, not factual correctness of the authorized value or faithful display to a reviewer.

6. Formal–concrete projection and executable conformance

An executable abstraction connects persisted admission events to the relational model:

$$\alpha : \text{PersistedPrePost} \rightarrow \text{BatchAlloyInstance}. \quad (21)$$

6.1. Persisted-state projection

For one selected event, α reads the persisted relations in table 6, the version chain, and pre/post identity sets. Canonical JSON equivalence classes become Alloy Value atoms; observed certificate identities become primitive `CertId` atoms. A decision and its binding become one `Authorization`. The mapping does not prove absence of digest collisions outside the observed instance.

For a committed case, an exact Alloy wrapper fixes the instance and tests `legalAdmission[event, Full]`. For a rejection, equal canonical database digests are required before generating a wrapper that checks state stutter and failure of `Full`. A changed digest fails projection before Alloy runs.

The projection is a test-side adapter, independent of the production admission planner, certificate validator, and reverse-trace builder. Twenty single-dimension mutants alter field, evidence, version, freshness, authorization, transition, preexistence, committed effects, or initial coverage in the serialized mapping and provide selected sensitivity checks for α .

6.2. Independent finite catalogue

A second oracle directly reads all SQLite application tables and checks value/source domains, source evolution, transition sets, version anchors, authorization lineage, and value agreement. Its expected results do not come from the production planner or trace builder. Five relation mutants check sensitivity to source, authorization, locator, principal, and value changes.

The declared 35-case catalogue contains five legal cases and thirty rejection, post-persistence corruption, schema-prevention, stateful-evolution, or oracle-mutant cases. Rejection cases require the public error classification and equality of a deterministic digest covering schema version, identity sequences,

Table 6: Principal elements of the executable abstraction.

Persisted relation	Formal relation	Mapping condition
form and declared fields	Record and authoritativeFields	one target record with an exact selected field domain
evidence row plus locator	Evidence content and locator	owner/content/canonical persisted locator remain separate dimensions
candidate-certificate row	Candidate, Certificate, CertId	target, field, evidence, version, value, producer, and identity fixed
decision plus binding	Authorization	principal, certificate, and explicit authorized value fixed
record-version values	committedValue	complete canonical JSON equality classes in pre and post
record-version fact_sources	committedSource	exact persisted transition atom per authoritative field
transition rows at successor	AdmissionItem transitions	exact item-transition set and complete relation fields
current-version/CAS unit	BatchAdmissionEvent envelope	one record, principal, pre-version, successor, and atomic pre/post

and every application table. Corruption cases instead require a changed digest and an incomplete trace/oracle diagnosis. The machine-readable declaration fixes the catalogue’s completeness scope.

6.3. Value-equality repair and witness-domain control

The repaired implementation uses canonical JSON equality for changed-field decisions. Thirteen regressions cover unchanged controls, numeric/Boolean and nested changes, and a rejected no-op; seven fail before the repair and all pass afterward. The witness checker also separates schema and declared-item domains, retaining the five class pairs and adding an identity pair and copy-forward control. Supplement S2 records both repairs and their checks.

6.4. Refinement boundary

Nine projected cases cover singleton Accept/Correction, multi-field all-Accept/all-Correction/mixed batches, initial coverage, copy-forward, stale rejection, and whole-batch rejection caused by one invalid item. These provide selected bounded refinement evidence, not a refinement proof for every application execution.

Two intentional scope differences remain. First, the concrete post-initial no-op rule narrows the formal same-value behavior. Second, SQLite uniqueness makes one modeled duplicate-source state unreachable through the trusted schema. Conversely, concrete trace tests cover persisted pointer and binding corruptions that the formal P6 tamper event does not encode. The formal `traceComplete` predicate is a single-level anchor and uniqueness condition over the persisted relations, whereas the concrete trace builder additionally traverses each predecessor prefix and retains exact previous sources. The bounded positive assertion therefore establishes the encoded admission step under its stated well-formed-pre-state condition, not prefix-wide trace completeness.

7. Evaluation protocol

7.1. Research questions and evidence status

The evaluation separates relational correctness, the boundary at which review becomes authorization, and implementation cost:

- RQ1** Do the formal and concrete checks support the admission contract, and what changes when a rich event journal records the exact reviewed candidate?
- RQ2** Which inconsistencies between recorded review intent and confirmation can server-held session binding reject, and which remain outside its guarantee?
- RQ3** What admission, reverse-trace, and storage costs occur on the repaired implementation under the declared workloads?

We retain the earlier bounded analysis and concrete checks as supporting evidence and add three experiments, E1–E3, corresponding to these questions. The new experiments use the equality-repaired implementation at source commit `c6d512843c905cab6d8521dd8c914f7fb26d85ae`, under `latest/code/implementation-fixed`. The protocol was written before collection on

23 September 2026, with documented pilot corrections; it was not externally preregistered. Source hashes, scripts, pilot outputs, final receipts, database snapshots, and per-trial measurements are deposited. No hosted model was called for E1–E3. Historical hosted-agent runs provide separate integration context and are not counted as new security experiments.

7.2. RQ1: correctness and comparison with a rich event journal

Retained checks. The bounded analysis executes 36 commands in each of two Alloy profiles. Concrete-to-formal checks cover nine intended projections and twenty mapping mutants. A 35-case production-facing catalogue distinguishes successful admission, zero-write rejection, and detection of post-persistence corruption. Stateful tests compare an independently maintained value/source model with persisted snapshots and traces; rollback and concurrency tests exercise transaction failpoints and contending calls. Their original manifests and execution environments remain identified separately in the supplement. They are not all claimed to have been rerun for the new measurements. Before E1–E3, 85 relevant implementation regression tests and ten experiment-harness checks were rerun.

E1 mechanisms and policies. We wrote a separate `sqlite3` event-journal comparator for this study. Both its configurations retain immutable candidates and events, review context, version checks, transactional updates, and complete field-source maps. *Context journal* records the field, version, evidence, and retained review context but omits the exact reviewed-candidate identity. *Exact journal* additionally binds that identity. The third arm calls the reference implementation’s actual transactional admission path. The comparator is a same-study implementation, not an externally deployed system or an independent-team replication.

We score two policies. The *instance policy* requires the authorized candidate instance; the *value/context policy* permits a different instance with the same retained review observations. The latter still checks field, version, evidence, and context, rather than value equality alone. Its projection omits certificate identifier, internal candidate identifier, and instance creation timestamp. Reporting both policies makes the cost of stricter identity requirements explicit.

Inputs and cases. Twelve archived proposal cells are selected deterministically across three historical configurations and four benign scenario labels: the

first lexicographic run containing a proposal is used, without selecting on task success. The selected configurations yield the same proposal in each scenario, giving only four distinct archived values: 8, 100, B-008, and 9. Three synthetic controls add integer/float, integer/Boolean, and nested numeric-type boundaries. Thus the fifteen inputs contain seven distinct proposal values. They are mapped into constructed form histories with newly prepared certificates and review events, not replayed as complete original workflows. Evidence substitutions manipulate constructed metadata rather than real image contents.

Each input exercises eleven case families: Accept, Correction, multi-field admission, same-value candidate substitution, different-evidence substitution, stale authorization, replay, unauthorized value, and interruption after CAS, before transition insertion, or before commit. This gives $15 \times 11 \times 3 = 495$ executions. The three interruption families examine rollback locations, not three independent adversary models.

Oracles. An independent database reader checks proposal value, authorized value, reviewer, reviewed candidate, current value, and source version for each of three fields after successful admission: eighteen answers per accepted case. Answers are classified as correct, wrong, ambiguous, or unavailable. Rejection compares logical fact-database digests immediately before and after admission. The starting point is *after authorization preparation*: the journal may already have persisted review intent, while the reference adapter captures the service-generated admission bundle before submitting it to the actual transaction. Zero-write rejection therefore concerns fact admission, not erasure of prior review events. Error reasons are checked against expected constraints or injected interruptions, rather than treating any exception as success. The original strengthened-baseline cases are rerun separately as continuity checks and excluded from the 495-execution denominator.

7.3. RQ2: review-to-confirmation boundary

E2 uses local Chrome and Playwright with a newly instrumented review-page fixture backed by the original `ReviewForms.confirm` service. An independent driver reads the displayed candidate identifier, proposal, and authorized value from the DOM before the authorization click. We compare the original confirmation path with an experimental server-held session gate binding candidate, version, reviewer, and authorized value.

Three JSON inputs (100, 2.5, and `true`) cross ten interactions and two paths, giving sixty browser cases. Six initial confirmations prepare replay cases and are additional setup operations. Accept, Correction, and legitimate reselection are positive controls. Candidate, authorized-value, and principal request substitutions test agreement with recorded review intent. Stale state, replay, and precommit interruption test rejection. A DOM-only substitution changes the displayed candidate while leaving the server and request target unchanged. The decimal input avoids JavaScript serializing an integral float as an integer.

We record agreement with the observed review, acceptance, and fact writes on rejection. This is a scripted boundary experiment, not a security evaluation of the original product UI or a human-subject study. Fixed trusted principal labels replace authentication. The gate’s session database is separate from the fact database; the experiment does not establish cross-database crash atomicity, hostile-host resistance, or human attention.

7.4. RQ3: repaired-version cost characterization

E3 ran on an Intel Core i7-10875H machine with Windows 10 build 19045, CPython 3.11.16, SQLite 3.53.1, and SQLAlchemy 2.0.51. Operating-system background activity was not controlled. Each comparison cell uses five warmups and 200 timed executions per arm, randomly interleaved with seed 20260923. Two ten-cell admission comparisons contribute 4,000 observations each; a 36-cell trace comparison contributes 14,400. The total is 22,400 timed observations, excluding warmups. These repeated measurements do not represent independent deployment samples. Historical pre-repair timings are not pooled with E3 or used to calculate its ratios.

Admission. The ten cells use 1, 8, 32, or 128 fields, with one, four, or all fields changed where distinct and feasible. The first comparison measures complete reference and exact-journal confirmation on a shared workload with recognition candidates registered before confirmation. Result queries must agree, including retained sources for unchanged fields. Admission uses SQLite DELETE journaling and FULL synchronization.

The timer covers the confirmation call, excluding database cloning and adapter construction. The journal opens its connection eagerly during construction; the reference ORM obtains its first connection lazily inside the call. Schema, ORM, and certificate-validation work also differ, and the journal does not implement every reference-application feature. Consequently this

is an interface-boundary cost characterization, not a connection-controlled microbenchmark, a feature-identical system competition, or an isolated measurement of binding overhead. Different SQL hooks count trigger execution differently, so absolute cross-implementation SQL totals are not compared.

The second comparison is a separate prevalidated-materialization ablation using manual sources. A full admission generates a valid relational delta outside the materialization timer. Applying this delta retains complete snapshots, source maps, authority rows, transactionality, and CAS, but moves validation and planning outside the timed region. Each resulting state is checked against the precomputed reference fingerprint. Generated review identifiers and timestamps preclude a claim of byte-identical databases across independently executed full admissions. The two comparisons use different source workloads; their full-path columns must not be cross-subtracted.

Trace and storage. The trace grid crosses 1, 8, 32, and 128 fields; 1, 10, and 100 versions; and 1, 100, and 1,000 total records. Both arms use the same current verifier. The point-read arm replaces three bulk repository reads—certificates, certificate–evidence associations, and transitions—with identifier enumeration and actual point lookups. Valid unrelated forms are seeded once and copied as database snapshots outside timing. After every invocation, the complete returned trace object must equal the counterpart result. This isolates an access-pattern change within the current verifier, not a reconstruction of the historical implementation.

Storage fixtures directly populate full and reduced schemas at 1,000, 10,000, and 100,000 transitions, using WAL/NORMAL and checkpointing before measuring the main database. They use single-field version growth and stable evidence references. The measurement excludes external images and model-output files. Direct population characterizes schema footprint; it neither demonstrates successful admission of every row nor makes the reduced schema audit-equivalent.

7.5. *Historical hosted-agent integration context*

The archived matrix contains $3 \times 3 \times 14 \times 10 = 1,260$ planned executions across three qualified configurations, three prompt variants, fourteen scenarios, and ten repetitions. The historical configurations are G1 (`gpt-5.6-luna`), G2 (`gpt-5.6-terra`), and D1 (`deepseek-v4-flash`); endpoint names are meta-data, not immutable weight identifiers. A pilot qualification rule preceded the matrix; all planned final runs, including runtime failures, were retained. The

exposed tools allowed proposal and verification, with confirmation reserved for the host.

For 360 benign runs, the original strict criterion requires specified exact call sequences; a post-hoc endpoint sensitivity permits extra calls while requiring the intended proposal and verification outcome. The stale scenario’s strict criterion also requires the word `stale`. Behavior-evaluable runs have sufficient evidence for scoring regardless of task success. Planned-run yield and conditional completion are reported separately; unscored cases receive no imputed verdict.

Host authority checks are a different endpoint: 720 admission attempts use fixed host-constructed invalid tuples, while 179 capability-unavailable checks make no admission call. Neither branch derives its challenge parameters from the model answer. Unauthorized mutation means failure to reject when rejection is expected or a changed authoritative-state digest. Their counts describe executed integration checks, not model-generated attack efficacy or population failure probabilities. Supplement S5 summarizes historical endpoints and annotation; original prompts, qualification and scoring rules, missingness, and author-involved annotation records remain in the artifact.

8. Results

8.1. *RQ1: contract checks and exact-binding equivalence*

Retained formal and concrete evidence. All 72 Alloy outcomes matched their command manifests. Each profile yielded six legal witnesses, eleven conjunct-removal witnesses, three trace-attack witnesses, and two witnesses for the positive trace assertion’s nonempty precondition; fourteen assertions had no counterexample within scope. Each removal admitted a malformed effect that the full relation rejected with a stuttering post-state. These results establish bounded encoding sensitivity; the conditional observation-omission argument is a separate result.

The concrete-to-formal adapter produced nine satisfiable intended instances and twenty unsatisfiable mapping mutants. The 35-case concrete catalogue passed, as did the two stateful profiles and 33 declared roll-back/concurrency tests in the retained execution. The illustrative lifecycle preserved a proposed quantity of 100 and its independently authorized correction to 101, copied the other two fields’ sources, and produced a complete reverse trace with matching export. Six archived AI-origin lifecycle cases and ten pinned-interface controls also matched their declared outcomes. These

Table 7: E1: 165 executions per mechanism. Instance violations are accepted cases violating the instance policy. Extra rejections are rejected cases valid under the value/context policy. Query counts refer to field-level answers after acceptance. All arms had zero value/context-policy violations and zero rejections with fact writes.

Mechanism	Accepted	Instance	Extra	Query answers	
	cases	violations	rejections	Correct	Ambiguous
Context journal	60	15	0	1,020	60
Exact journal	45	0	15	810	0
Reference	45	0	15	810	0

are distinct denominators, not additive independent attack trials. The new run passed all 85 selected implementation regressions and ten harness checks.

E1: same contract, different realizations. All 495 new case executions completed without harness errors. Table 7 reports the three mechanisms. The exact journal and reference agreed on all 165 paired decisions and query-answer sets. Each accepted all 45 legal cases and rejected 120 cases without modifying the fact database after authorization preparation. This includes the three injected rollback locations.

The context journal accepted fifteen same-value instance substitutions. They satisfy the declared value/context policy but violate the instance policy. Conversely, the exact mechanisms reject these fifteen otherwise value/context-valid substitutions. Thus the result is a policy distinction, not an unqualified claim that additional rejection is always preferable. All mechanisms rejected substitutions with different retained evidence; richer context already distinguishes those cases.

Across successful histories, the context journal returned 1,020 correct answers and sixty ambiguous reviewed-candidate answers. Each exact mechanism returned 810 correct answers and no ambiguous, wrong, or unavailable answer. The unequal totals follow from different numbers of accepted histories, each contributing eighteen field-level answers. They are neither independent query workloads nor estimates of real-world ambiguity prevalence.

The continuity run reproduced seven-of-eight single-history agreement for the original strengthened baseline, with same-value substitution as the difference; its paired-history distinction was a separate ninth check. The new exact-journal result qualifies that separation: event journals can satisfy the tested contract when they preserve the same authorization relation. E1

Table 8: E2: accepted cases by interaction family, with three JSON values per named interaction. Each path has thirty cases; replay setup is additional. All nine original-path rejections and eighteen gate rejections made zero fact writes.

Interaction family	Cases/path	Original	Session gate
Accept, Correction, reselection	9	9	9
Candidate, value, principal request substitutions	9	9	0
Stale, replay, precommit interruption	9	0	0
DOM-only substitution	3	3	3
All cases	30	21	12

therefore supports explicit information and enforcement requirements without establishing a uniquely capable storage architecture.

8.2. RQ2: request binding and the remaining display assumption

Both paths completed all thirty assigned browser cases and accepted all nine legitimate controls, including corrections and reselections (table 8). The original confirmation path accepted all nine request substitutions that disagreed with recorded review intent. The experimental gate rejected all nine. Stale submissions, replays, and precommit interruptions were rejected by both paths without fact writes.

The original service constructs its authorization bundle from the caller-selected candidate and value. The substitutions expose that trusted-caller boundary; they do not show a violation of the transaction’s subsequently established binding. Moving the binding to a server-held review session closes the tested request-substitution gap.

Both paths nevertheless accepted all three DOM-only substitutions: the page displayed candidate B while the server and request continued with A. Session binding therefore does not attest what the user saw. The negative controls delimit the claim to fidelity between a trusted review record and admission; display integrity and authenticated human intent remain premises. Scripted page observations cannot establish user attention or resistance to a compromised browser.

8.3. RQ3: current implementation costs

Complete confirmation and prevalidated materialization. The two admission comparisons completed all ten cells each. Final queries agreed for the shared

Table 9: E3 admission: p50/p95 in milliseconds, with 200 timed observations per cell and arm. The two comparison groups use different source workloads. Their full-path columns are not interchangeable; the prevalidated arm omits timed validation and planning.

Fields	Changed	Recognition-source mechanisms		Manual-source ablation	
		Reference	Exact journal	Full	Prevalidated
1	1	28.79/50.50	10.40/13.32	23.23/33.76	9.97/14.63
8	1	33.99/55.40	11.19/15.18	23.87/65.83	10.40/15.05
8	4	57.97/103.89	11.88/16.47	36.68/57.24	11.48/19.46
8	8	75.71/129.14	21.51/26.31	50.23/97.50	11.38/17.22
32	1	34.75/54.66	11.41/15.90	27.00/45.75	11.00/16.52
32	4	55.32/101.11	23.89/31.17	44.23/97.98	17.31/23.66
32	32	187.24/292.72	13.07/17.77	122.68/200.00	13.89/22.61
128	1	29.35/57.56	11.33/14.76	29.10/48.08	11.08/17.21
128	4	44.39/72.72	11.61/16.75	38.33/87.89	11.34/17.19
128	128	712.00/1039.83	22.09/30.53	427.59/621.20	23.50/37.43

mechanism workload; materialization results matched their precomputed relational-state fingerprints. Table 9 reports every cell, keeping the two workloads separate.

At 128 fields with all fields changed, median reference confirmation was 712.00 ms and exact-journal confirmation was 22.09 ms. This large implementation difference is evidence against presenting the reference as the fastest way to enforce the contract. Connection lifecycle, ORM, schema, and validation differences preclude attributing the gap to exact binding, which both mechanisms perform. The small-workload comparison is also affected by lazy versus eager connection establishment.

In the independent manual-source ablation at the same field/change counts, full confirmation had a median of 427.59 ms and prevalidated materialization 23.50 ms. Materialization removes validation and planning from the timed region and uses a different persistence path. The difference describes the work excluded by that protocol together with implementation-path effects; the materializer is not a complete secure admission alternative.

Reverse trace. The 36-cell paired comparison completed 14,400 timed invocations. Every full returned object agreed across the two access patterns. Bulk tracing used exactly twelve SQL statements in every measured cell; this bounds round trips for the tested implementation, not total verification time or memory.

Table 10: E3 trace: representative cells with 128 fields and 1,000 total records. Latencies are p50/p95 milliseconds; each arm has 200 timed observations per row. SQL counts are constant within each row and arm. The complete 36-cell grid is deposited.

Versions	Latency (ms)		SQL statements	
	Bulk	Point reads	Bulk	Point reads
1	28.68/43.26	209.81/256.03	12	396
10	39.69/48.16	233.82/285.43	12	423
100	170.24/216.75	509.09/595.85	12	693

Table 11: E3 synthetic schema footprint after checkpointing. Incremental bytes per transition equal the full-minus-reduced main-database size divided by transition count. External evidence files are excluded.

Transitions	Full (MiB)	Reduced (MiB)	Incremental bytes/transition
1,000	1.52	0.48	1085.4
10,000	13.16	2.88	1077.2
100,000	134.53	28.45	1112.3

At 128 fields, 100 versions, and 1,000 records, bulk tracing had p50/p95 of 170.24/216.75 ms, versus 509.09/595.85 ms for point reads, with twelve versus 693 SQL statements (table 10). The point-to-bulk median ratio was 2.99. Both arms are current-code runs with the same verifier, so this ratio concerns their access patterns rather than an improvement over historical timings. As history grows, processing and validating the returned data still increases despite constant round trips. Supplement S4 lists all 36 cells; raw observations and paired summaries are deposited.

Schema footprint. The full schema occupied 1.52, 13.16, and 134.53 MiB at the three tested scales, with roughly 1.1 kB of incremental main-database storage per transition relative to the reduced fixture (table 11). This accounting covers the stated synthetic relations and excludes external evidence files. It does not imply equivalent functionality in the reduced database or a universal per-transition storage constant.

8.4. Historical integration: task outcomes and separate host operations

The archived matrix retains all 1,260 planned executions. Strict benign-task yield was 320/360 (88.89%); endpoint yield allowing extra calls was

331/360 (91.94%). Twenty-five benign runs were unscorable. Conditional on the 335 behavior-evaluable runs, the counts were 320/335 and 331/335. The eleven differences reflect extra calls accepted by the endpoint criterion; no strict success failed that criterion. These are alternative measurements of the same traces, not additional successful trials. They do not support a population model ranking.

Runtime failures numbered 93: 76/420 for D1, 4/420 for G1, and 13/420 for G2. Failures were retained; some still had complete host-authority evidence. The 720 fixed invalid-tuple admission calls recorded no unauthorized authoritative mutation. The 179 capability-unavailable checks also recorded none, but made no admission call. Their sum is accounting only and is not reported as a model-generated attack success rate. The archival results demonstrate compatibility with a restricted proposal/verification interface, while E1 and E2 separately examine the admission and review boundaries.

9. Discussion and limitations

9.1. *What the contract adds to existing database mechanisms*

The admission relation specifies the information that must survive the transition from a proposal to an operational fact. Transactions provide an atomic boundary; compare-and-swap protects a predecessor; provenance records derivation; and authorization identifies permitted actions. None of these descriptions, taken alone, decides whether approval applies to a value, to a recorded context, or to one exact candidate instance. The contract makes that policy explicit and connects it to the construction and later interpretation of a complete record version. In the running example, the database must retain that the machine proposed 100, the human authorized 101, and an unchanged field retained its prior source.

The event-journal comparison is evidence of representational portability within the tested cases. Adding the exact review link produces the same 165 decisions and corresponding query answers as the reference implementation. This result prevents an architectural exclusivity claim: a journal can satisfy the contract, and a conventional schema can encode it through suitable keys, constraints, and transactions. The contribution is the stated relation, its conditional observation analysis, and the evidence linking enforcement to persisted outcomes. It is not a new transaction primitive or a demonstration that event sourcing is intrinsically insufficient.

Instance accountability is also a policy choice. The rich-context journal accepts 15 substitutions that preserve its declared value/context policy. Calling these errors without naming the stricter instance policy would prejudge the comparison. Conversely, the two exact-binding arms make 15 additional rejections under the looser policy. A deployment that treats equivalent extractions as interchangeable may accept that equivalence; one that must identify the exact reviewed artifact needs an explicit discriminator. Renewed approval without a changed value is a separate review-event operation because the concrete post-initial admission path creates transitions only for changed values.

9.2. Where authorization must be established

Candidate-bound storage preserves the authorization it receives. It cannot establish retrospectively which candidate a person actually saw. The original confirmation service constructs its authorization from the caller-selected certificate and value. The browser experiment shows why the trusted-caller assumption matters: request substitution can change that input before the database establishes a consistent binding. A server-held review session can bind an earlier review intention to the later request and rejects the nine tested request substitutions.

The three display-only substitutions accepted by both paths identify the remaining boundary. A server can receive the intended candidate while a manipulated page displays another one. The study therefore supports trusted presentation and authenticated-caller assumptions, rather than an end-to-end proof of human consent. The experimental session gate uses fixed principal labels and a separate session database; authentication, human attention, malicious-host resistance, and crash atomicity across the session and fact databases were not evaluated. The gate is an evaluated extension, not a property retroactively attributed to the original implementation.

9.3. Interpretation of cost and provenance reconstruction

The current measurements replace frozen v8 timings as the main cost evidence. They expose a substantial admission-cost difference between the reference implementation and the smaller exact-binding journal. The compared implementations differ in schema, ORM use, certificate validation, application functionality, and connection lifecycle. In particular, the journal connection is opened before the timer, whereas the reference ORM can establish its first connection within confirmation. The results describe these call boundaries; they

neither isolate the cost of candidate binding nor establish a feature-matched system ranking. The prevalidated-materialization arm omits validation and planning and is not a safe replacement for full admission.

The trace comparison holds the current verifier fixed and changes three repository operations from bulk access to identifier enumeration and point loads. Equality of the entire returned object is checked after each timed call. Thus the measured difference concerns data access within this verifier. Twelve SQL statements across the grid do not imply constant total cost: transferred rows, in-memory indexes, and predecessor traversal still grow with the selected record’s history and fields. Storage measurements characterize populated schemas; external evidence bytes, model-output files, and production retention policies are outside their footprint.

9.4. Formal scope and implementation validity

Proposition 1 is conditional on the declared history domain, observation projections, and outcome labels. In particular, an omitted observation cannot be recovered by decoding an opaque candidate handle. A different observation model can combine or derive these distinctions differently. The construction establishes neither universal minimality nor completeness over all admission failures. Alloy results apply to the encoded finite scopes, and several assertions check direct consequences of the definitions. The selected projection instances and mutants test the abstraction interface rather than prove a refinement theorem over every execution.

The reference implementation uses canonical JSON equality across change detection, authorized values, committed values, and tracing. The earlier integer/float and numeric/Boolean disagreement shows why change detection and provenance validation must share an explicit value-equivalence relation. The repaired regressions cover selected boundaries, not every serialization or migration behavior. The concrete prefix trace is stronger than the formal single-step anchor check in some respects; no-op admission and schema uniqueness also differ explicitly as described in section 6.

Trace completeness concerns persisted relations, not factual truth. Evidence comparison uses stored digests and canonical locators without re-reading external bytes. Out-of-band evidence replacement that leaves metadata unchanged requires immutable storage or an additional integrity check. Host authentication, serialization, policy state, hashing, and transaction execution remain trusted. Commit-entry policy revalidation rejects a revocation visible before that check but does not cancel a transaction revoked afterward. Other

database engines, distributed revocation, deletion, redaction, tombstones, and retention expiry require additional design and evidence.

9.5. Construct and external validity

The 495 executions are a factorial set of constructed cases, not 495 independent applications. Twelve archived model/configuration–scenario cells contribute only four distinct proposal values; three synthetic type controls bring the total to seven. Their reuse checks deterministic admission semantics, not generalization across models. The event journal was separately implemented for this study, not supplied by an independent institution. Evidence contexts in these cases are constructed metadata, not a benchmark of image tampering.

The 60 browser cases use an instrumented local fixture rather than the full deployed interface or a user study. Fault counts are therefore coverage observations, not attack-success or user-error rates. The new studies make no hosted-model calls. Historical hosted-agent results are separate integration observations: task completion depends on the scoring criterion and missing runtime outcomes, while fixed host checks exercise restricted capabilities rather than autonomous model-generated attacks. Their original labels, failures, and author-involved annotations remain archived (Supplement S5).

Timing repetitions come from one Windows machine with uncontrolled background activity. Interleaving limits ordering effects but cannot remove scheduling, caching, or connection-lifecycle differences. Repetition-based intervals, where provided in the artifact, describe that run rather than uncertainty across machines or deployments. A separately implemented application with different records, reviewers, and operational policies would provide the most useful next test of transferability.

9.6. Applying the relation

A practical adoption starts by choosing the approval policy and tracing one correction through persisted records. Identify where the reviewed candidate is selected, where the authorized value is fixed, how the transaction rejects a superseded predecessor, and how changed and unchanged fields acquire sources. Retain both a legal control and a failure probe at each boundary. Supplement S3 provides a compact worked example and inspection checklist. Passing those probes is evidence for the exercised path; extending the contract to an independent system remains a separate validation task.

10. Conclusion

Correction-aware data admission separates machine proposal, human value authority, and committed provenance. The contract binds an exact candidate to an explicit authorized value and a fresh predecessor, then atomically constructs a complete successor with sources for both changed and unchanged fields. Five paired histories establish conditional observation requirements, while bounded relational checks, selected persisted-state projections, and transaction tests connect those requirements to an executable realization.

The new experiments clarify both utility and limits. An exact-binding event journal matches the reference implementation on all 165 corresponding cases, locating the contribution in the admission relation rather than an exclusive storage architecture. An experimental review-session gate blocks the tested request substitutions, but display-only substitutions remain outside its guarantee. Current-version measurements expose implementation costs and show a benefit from bulk provenance access with the same verifier on the tested grid. These findings support an inspectable integrity boundary for AI-assisted data acquisition under explicit trust and workload assumptions. Independent deployments and operational review workflows remain to be evaluated.

Declarations

Data and software availability

The study uses synthetic and public lifecycle inputs. The new comparison, browser-boundary, and current-version cost evidence is deposited with code, raw receipts, database snapshots, screenshots, dependency locks, and manifests at <https://github.com/lhh666-6/auto-dete/tree/2645e5e18c900ea91c9c980e44195dc71e410432/DKE-supplement>. The reference implementation is fixed at commit `c6d512843c905cab6d8521dd8c914f7fb26d85ae`, under `latest/code/implementation-fixed`. Its formal models, observation witnesses, and earlier annotation and integration evidence are accessible from <https://github.com/lhh666-6/auto-dete/tree/r31-jss-2026-09-13/latest>. Historical v8 evidence remains separately versioned under tag `r21-jss-2026-09-07-v8`. Supplement S1 maps the evidence layers and describes restoration of the losslessly compressed large database. Inspecting the deposit and reproducing the three new experiments requires no paid model calls; repeating historical hosted-agent runs requires provider credentials.

Ethics approval and consent to participate

The new experiments use constructed data and scripted browser interactions, with no new human participants. In the historical annotation study, one author (X.W.) and one non-author volunteer independently coded benign completion outcomes; L.H. adjudicated the inter-human disagreement and separately reviewed eleven human–rule disagreements against the strict-trajectory criterion. X.W. also completed the textual-acknowledgment annotation. Only pseudonymous annotator identifiers were collected; both benign-endpoint coders were informed of the task and data use, and the non-author coder participated voluntarily. Annotation evaluated generated outputs rather than annotator characteristics. The authors assessed this activity as not requiring ethics approval. No animal subjects were involved.

CRedit authorship contribution statement

Liang Hanghao: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Validation, Visualization, Project administration, Writing—original draft, and Writing—review & editing.

Xuan Wentao: Methodology, Formal analysis, Investigation, Data curation, Validation, Visualization, and Writing—review & editing.

Chen Qile: Investigation and Validation.

Peng Peng: Supervision and Writing—review & editing.

Funding

This research received no external funding.

Declaration of competing interest

The authors declare no competing interests.

Declaration of generative AI and AI-assisted technologies

DeepSeek, OpenAI Codex, and Claude assisted earlier stages of code, analysis, figure, and manuscript preparation. OpenAI Codex assisted the present revision with supplementary experiment implementation and execution, evidence checks, literature organization, manuscript restructuring and drafting, and deterministic table generation and PDF compilation. The three new experiments use archived proposal values and synthetic inputs and make no hosted-model calls. Historical benchmark calls to two requested OpenAI configurations and one requested DeepSeek configuration are distinct research

data, with requested identifiers and raw events retained in the archive. Historical human annotation labels were supplied by their named human coders; AI review proposals do not replace those labels or constitute human adjudication. No AI-generated raster figure is included. Research interpretation, source verification, approval of the final manuscript, and responsibility for the work rest with the authors.

References

- Arab, B.S., Gawlick, D., Krishnaswamy, V., Radhakrishnan, V., Glavic, B., 2018. Using reenactment to retroactively capture provenance for transactions. *IEEE Transactions on Knowledge and Data Engineering* 30, 599–612. URL: <https://doi.org/10.1109/TKDE.2017.2769056>, doi:10.1109/TKDE.2017.2769056.
- Buneman, P., Khanna, S., Tan, W.C., 2001. Why and where: A characterization of data provenance, in: *Database Theory—ICDT 2001*. Springer Berlin Heidelberg. volume 1973 of *Lecture Notes in Computer Science*, pp. 316–330. URL: https://doi.org/10.1007/3-540-44503-X_20, doi:10.1007/3-540-44503-X_20.
- Cheney, J., Chiticariu, L., Tan, W.C., 2009. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases* 1, 379–474. URL: <https://doi.org/10.1561/19000000006>, doi:10.1561/19000000006.
- Cui, H., Tang, Z., Yao, Z., Meng, F., Ma, Q., Jia, W., 2026. MemTxn: A transaction boundary for source-supported updates and complete-state recovery in agent memory. URL: <https://arxiv.org/abs/2607.27834>, doi:10.48550/arXiv.2607.27834, arXiv:2607.27834. preprint.
- Debenedetti, E., Zhang, J., Balunovic, M., Beurer-Kellner, L., Fischer, M., Tramèr, F., 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents, in: *Advances in Neural Information Processing Systems*, Neural Information Processing Systems Foundation. pp. 82895–82920. doi:10.52202/079017-2636.
- Fett, D., Campbell, B., Bradley, J., Lodderstedt, T., Jones, M., Waite, D., 2023. OAuth 2.0 Demonstrating Proof of Possession (DPoP). RFC 9449. Internet Engineering Task Force. URL: <https://datatracker.ietf.org/doc/rfc9449/>, doi:10.17487/RFC9449.

- Fowler, M., 2005. Event sourcing. Architecture pattern article. URL: <https://martinfowler.com/eaDev/EventSourcing.html>. published 12 December 2005; accessed 26 September 2026.
- Gray, J., 1981. The transaction concept: Virtues and limitations, in: Proceedings of the Seventh International Conference on Very Large Data Bases, pp. 144–154. URL: <https://web.stanford.edu/class/archive/cs/cs240/cs240.1236/old/sp2014/readings/Gray81.pdf>.
- He, J., Yu, D., 2026a. Beyond memory: A transactional continuity kernel for long-lived AI agents. URL: <https://arxiv.org/abs/2608.11632>, doi:10.48550/arXiv.2608.11632, arXiv:2608.11632. preprint.
- He, J., Yu, D., 2026b. Stored is not supported: Typed provenance and assertion guardrails for persistent AI agents. URL: <https://arxiv.org/abs/2609.02127v1>, doi:10.48550/arXiv.2609.02127, arXiv:2609.02127. preprint, version 1, 2 September 2026.
- Jackson, D., 2002. Alloy: A lightweight object modelling notation. ACM Transactions on Software Engineering and Methodology 11, 256–290. URL: <https://doi.org/10.1145/505145.505149>, doi:10.1145/505145.505149.
- Kung, H.T., Robinson, J.T., 1981. On optimistic methods for concurrency control. ACM Transactions on Database Systems 6, 213–226. URL: <https://www.eecs.harvard.edu/~htk/publication/1981-tods-kung-robinson.pdf>, doi:10.1145/319566.319567.
- Marinov, D., Khurshid, S., 2001. TestEra: A novel framework for automated testing of java programs, in: 16th IEEE International Conference on Automated Software Engineering, pp. 22–31. URL: <https://doi.org/10.1109/ASE.2001.989787>, doi:10.1109/ASE.2001.989787.
- Moreau, L., Missier, P., 2013. PROV-DM: The PROV Data Model. W3C Recommendation. World Wide Web Consortium. URL: <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>.
- Saidi, W., 2026. MutMem: Cryptographically authorized mutation in persistent agent memory. URL: <https://arxiv.org/abs/2608.02843>, doi:10.48550/arXiv.2608.02843, arXiv:2608.02843. preprint.

- Torlak, E., Jackson, D., 2007. Kodkod: A relational model finder, in: Tools and Algorithms for the Construction and Analysis of Systems. Springer. volume 4424, pp. 632–647. URL: https://doi.org/10.1007/978-3-540-71209-1_49, doi:10.1007/978-3-540-71209-1_49.
- Zhou, H., Zhang, L., Fan, Y., He, T., Chen, S., Geng, H., Torr, P., Yin, Z., 2026. LatticeMind: A conflict-aware memory primitive for multi-agent systems. URL: <https://arxiv.org/abs/2608.08236>, doi:10.48550/arXiv.2608.08236, arXiv:2608.08236. preprint.